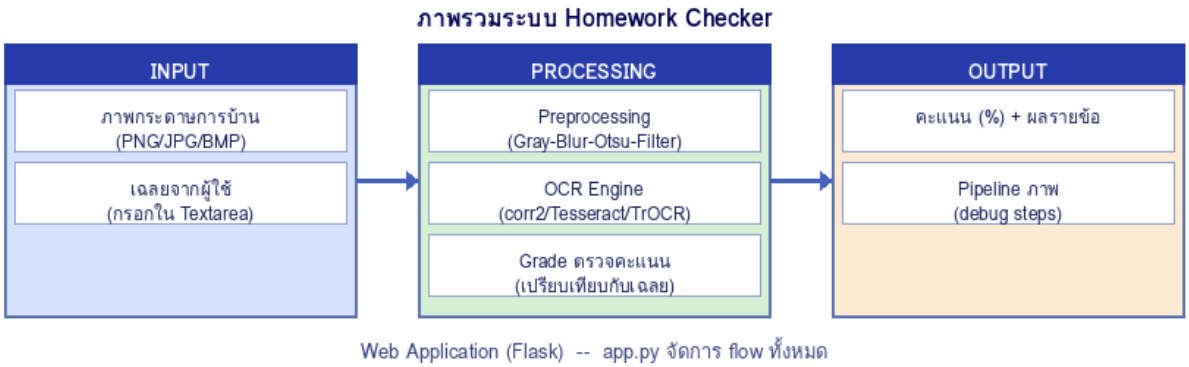
Project Name: Homework Checker — ตรวจการบ้านด้วย OCR

Creator: นายอัทธเมศร์ พิธาพัฒน์ธนกุล 2211311705

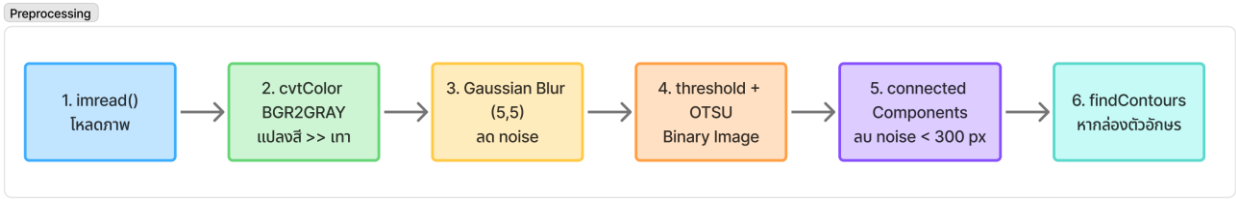
นายนวัตกรณ์ นามบุรินทร์ 2211311804

Descriptions: โปรแกรมตรวจการบ้านอัตโนมัติ รับภาพกระดาษการบ้านที่มีคำตอบเป็นตัวเลขหรือตัวอักษร แล้วใช้ 3 วิธีในการอ่านตัวอักษร ได้แก่ เลือก engine ได้จากหน้าเว็บ รองรับ Compare Mode (รัน 3 วิธีพร้อมกัน) และรับเฉลยจากผู้ใช้โดยตรง ใช้ 3 วิธี ได้แก่ **(1) Correlation 2D (corr2)** หลักการเดียวกับ MATLAB ของอาจารย์ **(2) Tesseract OCR** สำหรับตัวพิมพ์ภาษาอังกฤษและตัวเลข **(3) TrOCR** (Microsoft) สำหรับลายมือ ทำงานผ่านเว็บแอปพลิเคชัน Flask พัฒนาด้วย Python + OpenCV



ภาพรวมระบบ: ภาพกระดาษการบ้าน >> Preprocessing >> OCR >> ตรวจคะแนน >> แสดงผล

ขั้นตอน Preprocessing



ขั้นตอน Preprocessing ทั้ง 6 ขั้น

รายละเอียดแต่ละขั้น:

1. imread() — โหลดภาพจากไฟล์เป็น BGR array (3 channels) ด้วย OpenCV

2. cvtColor (BGR2GRAY) — แปลงภาพสี 3 channels ให้เป็น Grayscale 1 channel ค่า pixel 0–255 เทียบเท่า rgb2gray

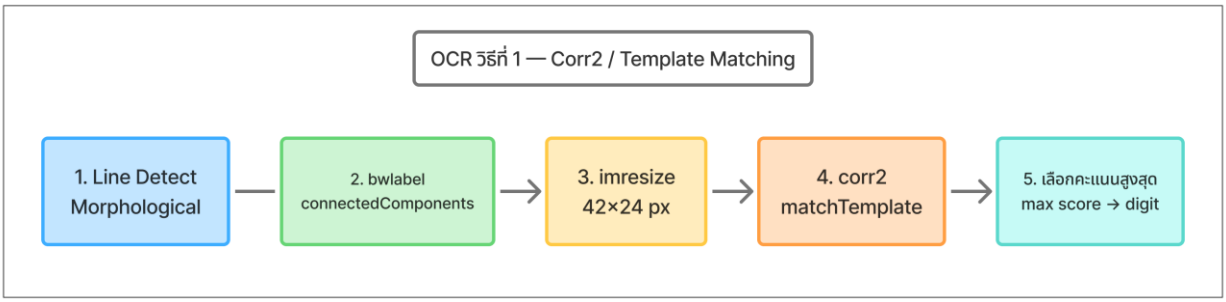
3. GaussianBlur (kernel 5×5) — เบลอภาพเล็กน้อยเพื่อกำจัด noise ระดับ pixel ก่อนทำ threshold หาก noise มีมากโดยไม่เบลอก่อน Otsu อาจเลือก threshold ผิดพลาด

4. Otsu Threshold (THRESH_BINARY_INV + THRESH_OTSU) — แปลงภาพ Grayscale เป็น Binary Image โดยอัลกอริทึม Otsu จะหาค่า threshold ที่แบ่งแยก pixel สองกลุ่ม (หมึก/กระดาษ) ได้ดีที่สุดโดยอัตโนมัติ ผลลัพธ์คือ pixel ตัวเลข = 255 (ขาว) พื้นหลัง = 0 (ดำ)

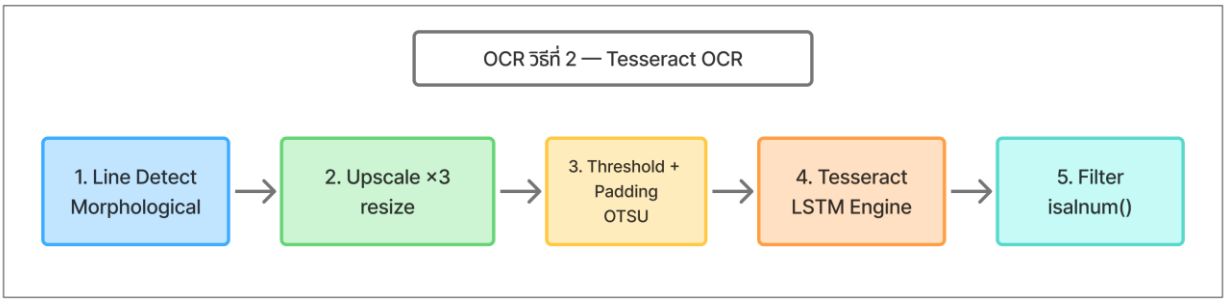
5. connectedComponentsWithStats (area ≥ 300 px) — วิเคราะห์ connected component ทั้งหมดในภาพ Binary แล้วลบกลุ่มที่มีพื้นที่น้อยกว่า 300 pixel ทิ้ง ช่วยกำจัด noise จุดเล็กที่อาจรบกวนการหากล่อง

6. Morphological Line Detection + findContours — สกัดเส้นแนวนอน (kernel 40×1) และแนวตั้ง (kernel 1×25) ด้วย MORPH_OPEN แล้วรวมเป็น box_mask จากนั้น MORPH_CLOSE (5×5, 2 รอบ) เชื่อมมุมที่ขาด กรองด้วย: พื้นที่ > 0.3% ของภาพ, ตำแหน่ง y > 5% ของความสูง (ตัด header), อัตราส่วน w/h อยู่ระหว่าง 1.0–15.0

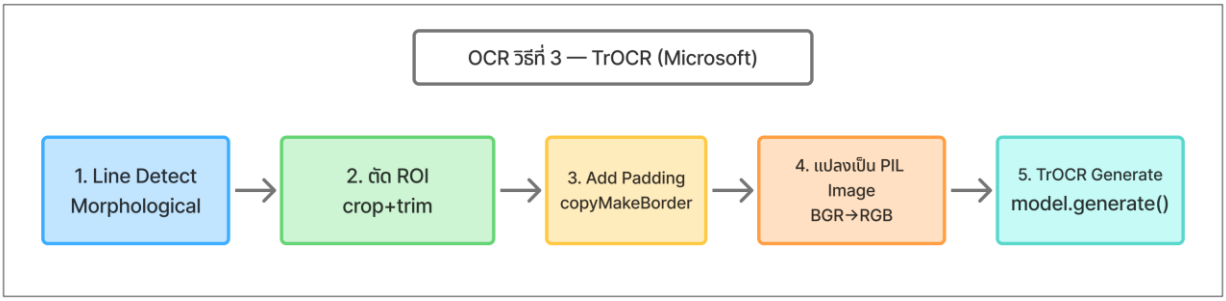
ขั้นตอน OCR วิธีที่ 1 — Corr2 / Template Matching



ขั้นตอน OCR วิธีที่ 2 — Tesseract OCR



ขั้นตอน OCR วิธีที่ 3 — TrOCR (Microsoft)



Conceptual:

1. แบ่งโปรแกรมเป็น 3 ส่วนหลัก

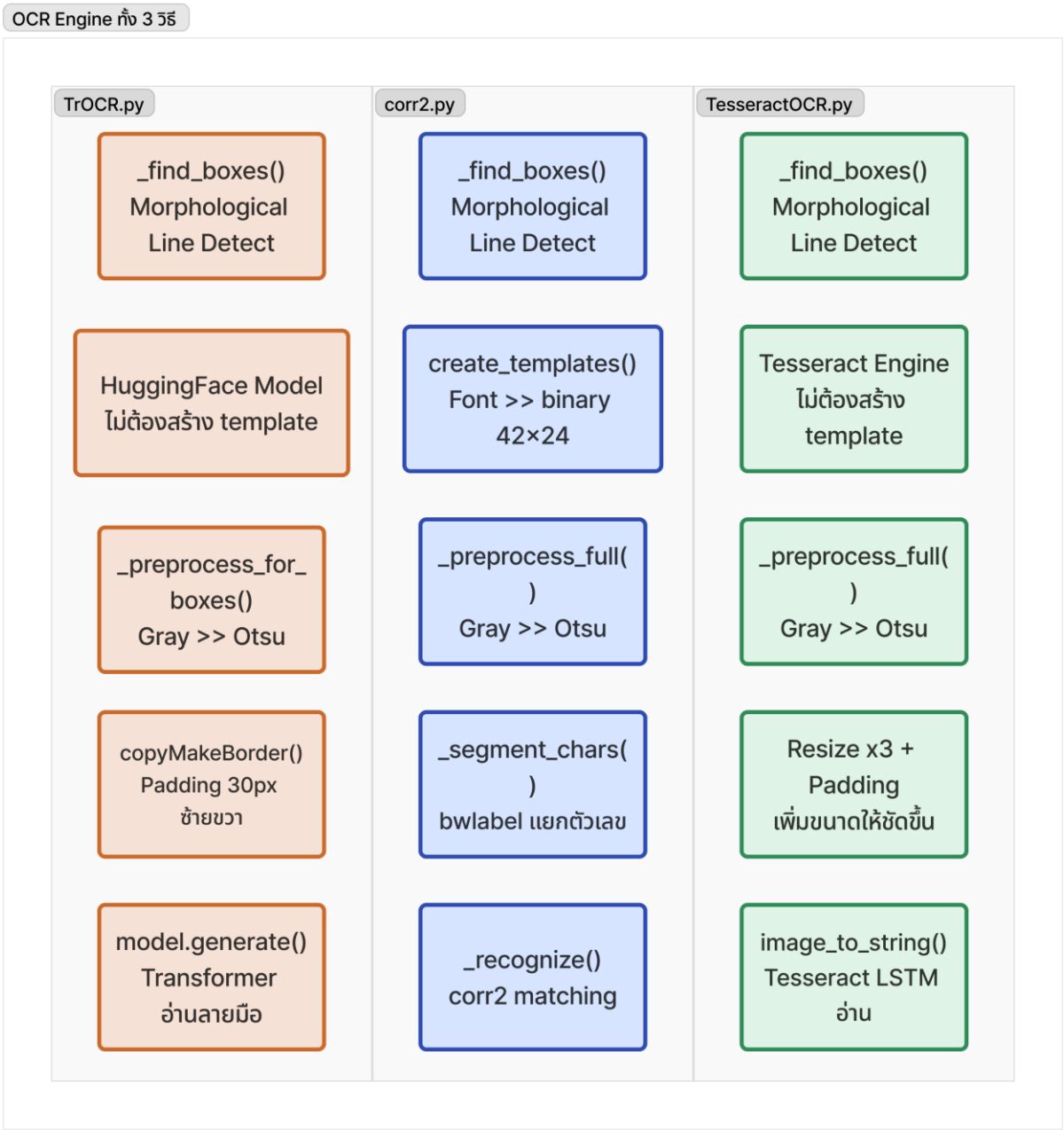
- app.py — Web Application (Flask) รับภาพและเฉลยจากผู้ใช้
- ocr_corr2.py, ocr_teserract.py, ocr_trocr.py — OCR Engine (3 ไฟล์แยกตาม engine)
- checker_corr2.py, checker_teserract.py, checker_trocr.py — Grading Logic (3 ไฟล์แยกตาม engine)

2. OCR Engine ทั้ง 3 วิธีใช้ Public API เหมือนกัน

- extract_answers(image_path) >> list[str]
- extract_answers_debug(image_path) >> (answers, pipeline_steps)

3. คำสั่งหลักที่สำคัญแต่ละวิธี

- ocr_corr2.py >> cv2.matchTemplate (TM_CCOEFF_NORMED)
- ocr_teserract.py >> pytesseract.image_to_string(roi, config='--oem 3 --psm 7')
- ocr_trocr.py >> model.generate(pixel_values, max_new_tokens=15)



สถาปัตยกรรม OCR Engine ทั้ง 3 วิธีเปรียบเทียบกัน

วิธีทดสอบโปรแกรม

0. Clone หรือ Download Project จาก Github : https://github.com/Nawattakorn/DIP-Project-HomeworkChecker-OCR

1. ติดตั้ง Dependencies ผ่าน terminal

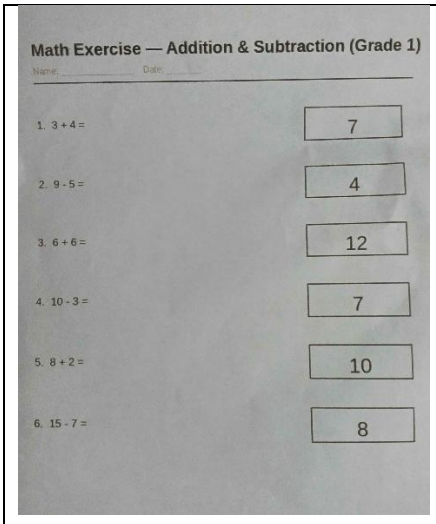
- Corr2: pip install flask opencv-python-headless pillow numpy werkzeug
- Tesseract: ติดตั้ง Tesseract-OCR Engine + pip install pytesseract

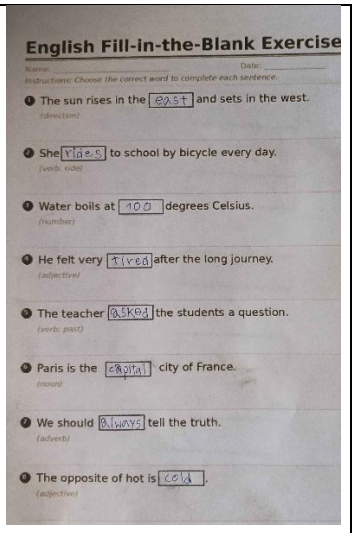
- TrOCR: pip install torch transformers

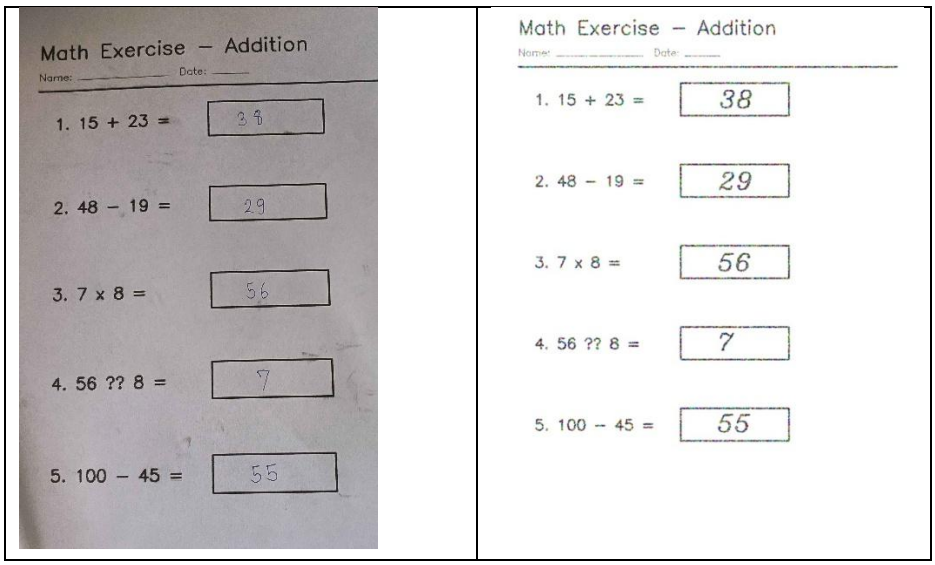
2. ไฟล์ในโปรเจค

```
source_code/
├── app.py              ← Flask web app (entry point)
├── ocr_corr2.py        ← OCR engine: Template Matching
├── ocr_teserract.py    ← OCR engine: Tesseract
├── ocr_trocr.py        ← OCR engine: TrOCR (AI)
├── checker_corr2.py    ← Grading logic สำหรับ corr2
├── checker_teserract.py ← Grading logic สำหรับ Tesseract
├── checker_trocr.py    ← Grading logic สำหรับ TrOCR
└── templates/
    ├── base.html       ← Layout หลัก (dark theme)
    ├── index.html      ← หน้า upload + เลือก engine
    ├── result.html     ← หน้าแสดงผลการตรวจ (single engine)
    └── compare.html    ← หน้าเปรียบเทียบ 3 engine
```

3. ภาพตัวอย่างสำหรับทดสอบ ใน folder: test_image







4. รัน Web Application

```
python app.py   # เปิดเบราว์เซอร์ที่ http://localhost:5000
```

/* ---------- */

/* Code: ocr_corr2.py — OCR วิธีที่ 1: Correlation 2D */

/* ---------- */

```
# ═══ 1. Template Creation (สร้างแม่แบบ 0-9) ═══════════════════════
def _make_template(char: str, font) -> np.ndarray:
    canvas = 128
    img = Image.new("L", (canvas, canvas), 255)
    draw = ImageDraw.Draw(img)
    try: draw.text((canvas//2, canvas//2), char, font=font, fill=0, anchor="mm")
    except Exception: draw.text((40, 40), char, font=font, fill=0)
    arr = np.array(img)
    rows, cols = np.any(arr < 200, axis=1), np.any(arr < 200, axis=0)
    if not rows.any(): return np.zeros((TMPL_H, TMPL_W), dtype=np.float32)
    r0, r1 = np.where(rows)[0][[0, -1]]
    c0, c1 = np.where(cols)[0][[0, -1]]
    crop = arr[r0:r1+1, c0:c1+1]
    resized = cv2.resize(crop, (TMPL_W, TMPL_H), interpolation=cv2.INTER_AREA)
    _, bw = cv2.threshold(resized, 127, 1, cv2.THRESH_BINARY_INV)
    return bw.astype(np.float32)

def create_templates():
    templates = {d: [] for d in DIGITS}
    loaded_fonts = []
    for fp in _FONT_CANDIDATES:
        try: loaded_fonts.append(ImageFont.truetype(fp, size=80))
        except Exception: continue
```

```
    if loaded_fonts:
        for font in loaded_fonts:
            for d in DIGITS:
                t = _make_template(d, font)
                if t.sum() > 0: templates[d].append(t)

    for d in DIGITS:
        if not templates[d]: templates[d].append(np.zeros((TMPL_H, TMPL_W),
dtype=np.float32))
    return templates

_TEMPLATES = create_templates()

# ═══ 2. OpenCV Correlation Engine  ═══════════════════
def match_score(img: np.ndarray, tmpl: np.ndarray) -> float:
    """คำนวณความเหมือนด้วย Normalized Cross-Correlation ของ OpenCV"""
    res = cv2.matchTemplate(img.astype(np.float32), tmpl.astype(np.float32),
cv2.TM_CCOEFF_NORMED)
    return float(res[0][0])

def _recognize(char_img, templates):
    if char_img is None or char_img.size == 0: return "?", -2.0
    resized = cv2.resize(char_img, (TMPL_W, TMPL_H), interpolation=cv2.INTER_AREA)
    if resized.dtype != np.uint8: resized = (resized > 0).astype("uint8") * 255
    _, bw = cv2.threshold(resized, 0, 1, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    bw = bw.astype(np.float32)

    digit_scores = {}
    for digit, tlist in templates.items():
        # ใช้ฟังก์ชันของ OpenCV ค้นหาคะแนนสูงสุด
        digit_scores[digit] = max(match_score(bw, tmpl) for tmpl in tlist)

    ranked = sorted(digit_scores.items(), key=lambda kv: -kv[1])
    best_label, best_score = ranked[0] if ranked else ("?", -2.0)
    second_label, second_score = ranked[1] if len(ranked) > 1 else ("?", -2.0)

    # แก้ปัญหา 2 คล้าย 3
    if best_label == "3" and second_label == "2" and (best_score - second_score) <
0.08:
        best_label = "2"

    return best_label, best_score
/* ---------------------------------------------- */
```

/* -- */

/* Code: ocr_teserract.py — OCR วิธีที่ 2: Tesseract */

/* -- */

```
import cv2
import numpy as np
import pytesseract

pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files\Tesseract-
OCR\tesseract.exe'

# ═══ Preprocessing ═══════════════════════════════════════════════
def _preprocess_full(img_bgr):
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
    gray_blur = cv2.GaussianBlur(gray, (5, 5), 0)
    _, bw_thresh = cv2.threshold(gray_blur, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)
    bw_raw = cv2.medianBlur(bw_thresh, 3)          # ← เวอร์ชันปัจจุบัน (มี medianBlur)
    num, labels, stats, _ = cv2.connectedComponentsWithStats(bw_thresh,
connectivity=8)
    bw_clean = np.zeros_like(bw_thresh)
    for i in range(1, num):
        if stats[i, cv2.CC_STAT_AREA] >= 300:
            bw_clean[labels == i] = 255
    return bw_clean, bw_raw, bw_thresh              # bw_thresh = ก่อน medianBlur

def _find_boxes(img_bgr, img_shape):
    """
    ใช้ Morphological Line Detection (เหมือน TrOCR) เพื่อหากล่องจาก "เส้นกรอบ"
    แก้ปัญหา MORPH_CLOSE เดิมที่ดูดข้อความนอกกล่องมารวมด้วย
    """
    h_img, w_img = img_shape[:2]

    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
    gray_blur = cv2.GaussianBlur(gray, (5, 5), 0)
    _, bw = cv2.threshold(gray_blur, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)

    # 1. สกัดเส้นแนวนอน (40px+ กันเส้น baseline สั้นใต้ตัวอักษร)
    hor_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (40, 1))
    hor_lines  = cv2.morphologyEx(bw, cv2.MORPH_OPEN, hor_kernel)

    # 2. สกัดเส้นแนวตั้ง (25px+ กันเส้นตัวอักษร)
    ver_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (1, 25))
    ver_lines  = cv2.morphologyEx(bw, cv2.MORPH_OPEN, ver_kernel)
```

```
    # 3. รวมเส้น → โครงกล่อง
    box_mask = cv2.add(hor_lines, ver_lines)

    # 4. เชื่อมมุมกล่องที่อาจขาด
    close_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    closed = cv2.morphologyEx(box_mask, cv2.MORPH_CLOSE, close_kernel,
iterations=2)

    contours, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    boxes = []
    for cnt in contours:
        x, y, w, h = cv2.boundingRect(cnt)
        area   = w * h
        aspect = w / float(h) if h > 0 else 0

        if area   < (w_img * h_img) * 0.003: continue  # กล่องเล็กเกินไปทิ้ง
        if y      < h_img * 0.05:            continue  # ขอบกระดาษบนทิ้ง
        if not (1.0 <= aspect <= 15.0):       continue  # ต้องเป็นสี่เหลี่ยมแนวนอน

        boxes.append((y, x, w, h))

    boxes.sort(key=lambda b: b[0])
    return boxes, closed
/* ------------------------------------------------ */
```

/* -- */

/* Code: ocr_trocr.py — OCR วิธีที่ 3: Microsoft TrOCR */

/* -- */

```
# ═══ 1. Line Detection Box Finder ═══════════════════
def _preprocess_for_boxes(img_bgr):
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
    gray_blur = cv2.GaussianBlur(gray, (5, 5), 0)

    _, bw = cv2.threshold(gray_blur, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)
    return bw

def _find_boxes(bw, img_shape):
    h_img, w_img = img_shape[:2]

    # 1. สกัดเส้นแนวนอน (40px+ กันเส้น baseline สั้นใต้ตัวอักษร)
    hor_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (40, 1))
    hor_lines = cv2.morphologyEx(bw, cv2.MORPH_OPEN, hor_kernel)

    # 2. สกัดเส้นแนวตั้ง (25px+ กันเส้นตัวอักษร)
    ver_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (1, 25))
    ver_lines = cv2.morphologyEx(bw, cv2.MORPH_OPEN, ver_kernel)

    # 3. เอาเส้นมาประกอบร่างกัน จะได้เฉพาะโครงสร้างกล่อง/ตาราง
    box_mask = cv2.add(hor_lines, ver_lines)

    # 4. เชื่อมมุมกล่องให้สนิทเผื่อเส้นขาด
    close_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    closed = cv2.morphologyEx(box_mask, cv2.MORPH_CLOSE, close_kernel,
iterations=2)

    # 5. หา Contours จากโครงกล่องเพียวๆ
    contours, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    boxes = []
    for cnt in contours:
        x, y, w, h = cv2.boundingRect(cnt)
        area = w * h
        aspect = w / float(h) if h > 0 else 0

        if area < (w_img * h_img) * 0.003: continue  # กล่องเล็กไปทิ้ง
        if y < h_img * 0.05: continue                # ขอบกระดาษด้านบนทิ้ง
        if not (1.0 <= aspect <= 15.0): continue     # ต้องเป็นทรงสี่เหลี่ยมผืนผ้าแนวนอน

        boxes.append((y, x, w, h))

    boxes.sort(key=lambda b: b[0])
    return boxes, closed
```

```python
# ==== 2. TrOCR Recognition ==================================================
def _recognize_trocr(roi_color):
    """ส่งภาพกล่อง (ที่เป็นสี) ไปให้ ตัว อ่านลายมือ"""
    roi_padded = cv2.copyMakeBorder(roi_color, 30, 30, 30, 30, cv2.BORDER_CONSTANT,
value=[255, 255, 255])
    roi_rgb = cv2.cvtColor(roi_padded, cv2.COLOR_BGR2RGB)
    pil_image = Image.fromarray(roi_rgb)

    pixel_values = processor(images=pil_image,
return_tensors="pt").pixel_values.to(device)
    with torch.no_grad():
        generated_ids = model.generate(pixel_values, max_new_tokens=15)

    return processor.batch_decode(generated_ids,
skip_special_tokens=True)[0].strip()
```

```
/* ---------------------------------------------- */
```

ผลการทดสอบ

ใช้วิธี Corr2 (Template Matching)

รองรับเฉพาะตัวเลข เนื่องจากมีความคล้ายกันของ Font Template จึงไม่นำตัวอักษรมาทดสอบ

ตัวเลขตัวพิมพ์

ตัวเลขลายมือ

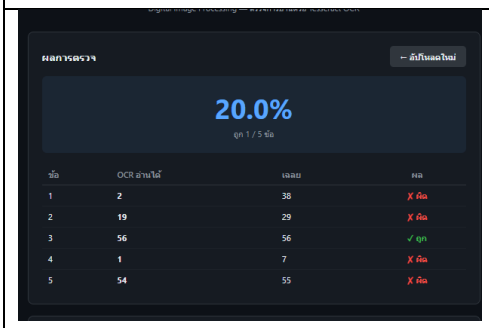
ผลพบว่า: ตัวเลขตัวพิมพ์ใช้ได้ดี แต่ลายมือยังใช้ไม่ได้ เนื่องจาก template สร้างจาก font ที่ต่างจากลายมือ

ใช้ Tesseract OCR

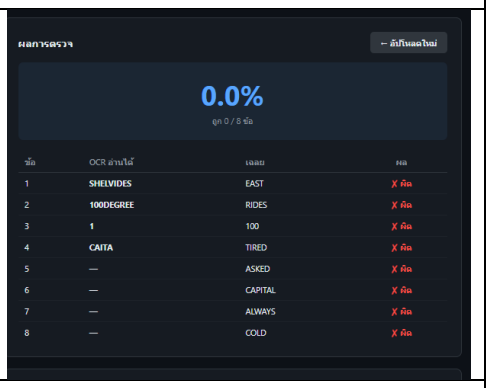
ทดสอบกับทั้งตัวเลขและตัวอักษรภาษาอังกฤษ ทั้งแบบตัวพิมพ์และลายมือ

ลายมือ

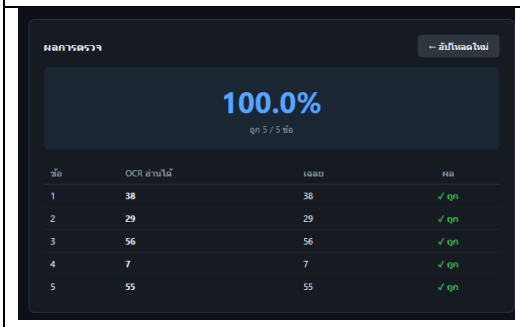
ตัวเลข

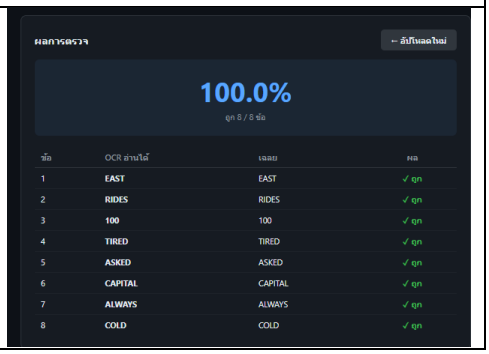


ตัวอักษร



ตัวพิมพ์



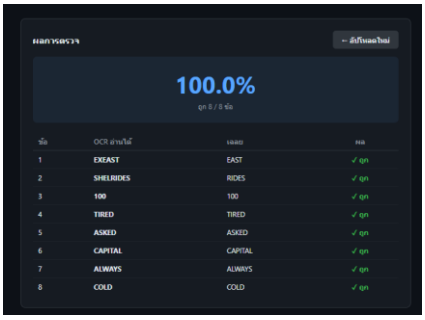


ผลพบว่า: ตัวเลขและอักษรตัวพิมพ์ทำงานได้ดี แต่ลายมือยังจับไม่ได้ เนื่องจาก Tesseract ถูกฝึกกับตัวพิมพ์เป็นหลัก

ใช้ TrOCR — Microsoft

ทดสอบกับลายมือเป็นหลัก เนื่องจากโมเดลถูก pre-train ด้วย handwritten dataset

ตัวเลขลายมือ



ตัวอักษรลายมือ

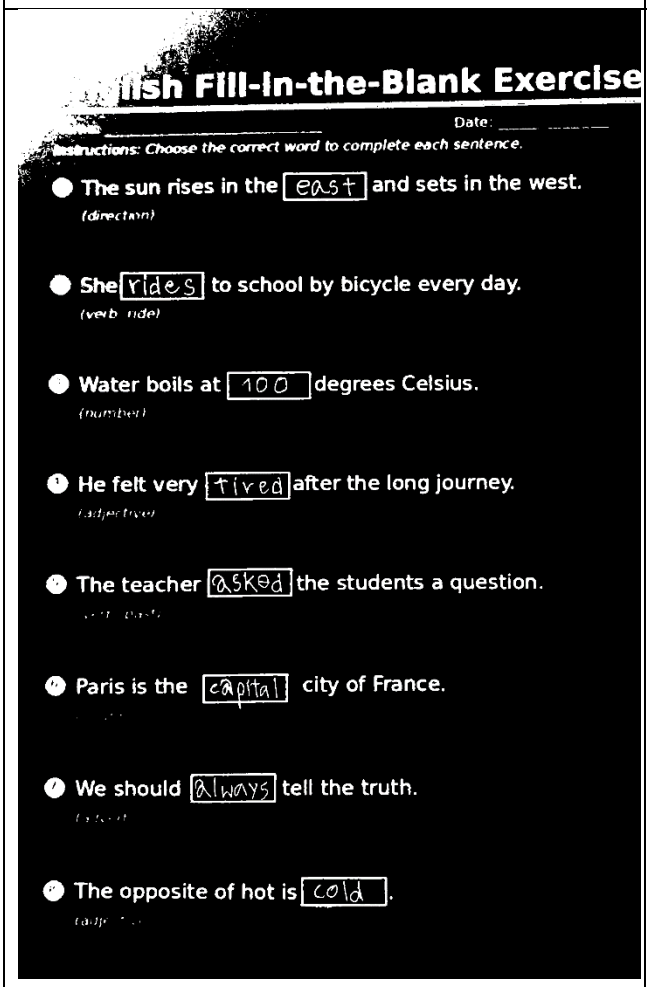
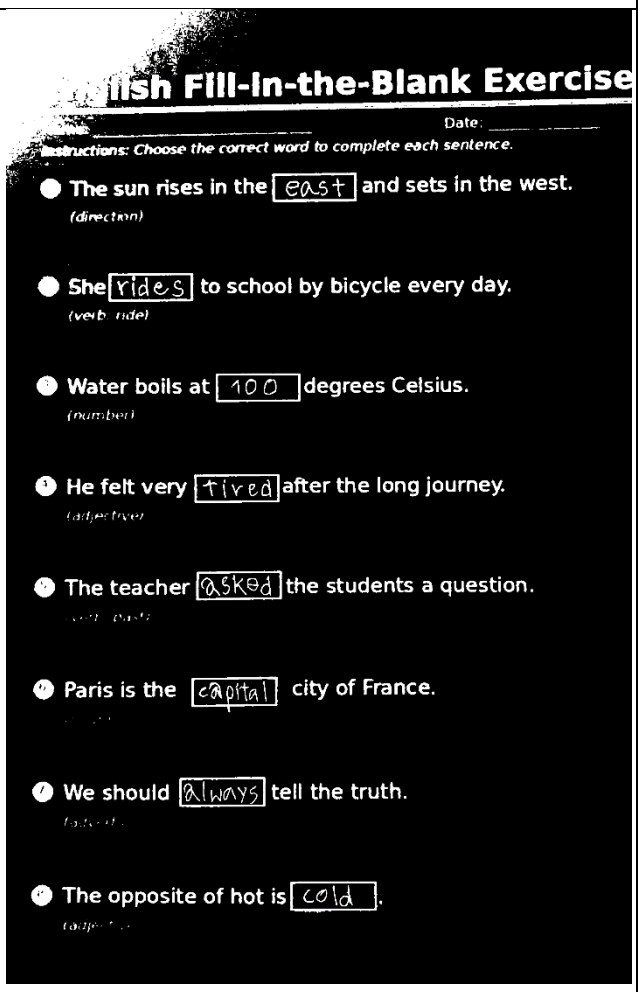
ผลพบว่า: TrOCR ทำงานได้ดีกับตัวอักษรลายมือ แต่ตัวเลขยังใช้ไม่ได้

ตารางสรุปผลการทดสอบ

คุณสมบัติ	Corr2 (Template)	Tesseract OCR	TrOCR (Microsoft)
หลักการ	Correlation 2D Template Matching	LSTM + CTC (Rule-based)	Transformer (Vision Encoder)
ประเภทที่รองรับ	ตัวเลข 0-9 (ตัวพิมพ์)	ตัวเลข + อักษร EN (ตัวพิมพ์)	ตัวเลข + อักษร EN (ลายมือ)
ตัวเลข (ตัวพิมพ์)	ดีมาก	ดีมาก	ดี
ตัวเลข (ลายมือ)	ไม่ดี	ไม่ดี	ไม่ดี
อักษร EN (ตัวพิมพ์)	ไม่รองรับ	ดีมาก	ดีมาก
อักษร EN (ลายมือ)	ไม่รองรับ	ไม่ดี	ดี
ความเร็ว	เร็วมาก (CPU)	เร็ว (CPU)	ช้า (ต้องการ GPU)

สรุปผลการทดสอบความแม่นยำของทั้ง 3 วิธีแยกตามประเภทข้อมูล

Feedback – ให้เปรียบเทียบการใช้และไม่ใช้ medianblur หลัง THRESHOLD

ใช้ medianblur	ไม่ใช้ medianblur
	

สรุปผลทดสอบการ ใช้ medianblur หลัง threshold นั้น และไม่ใช้ พบว่า ไม่มีความแตกต่างกัน

คำสั่งที่เรียนรู้และใช้งาน

คำสั่ง / ฟังก์ชัน	คำอธิบาย
cv2.imread(filename)	โหลดภาพจากไฟล์ — เทียบเท่า imread ใน MATLAB
cv2.cvtColor(BGR2GRAY)	แปลงเป็น Grayscale — เทียบเท่า rgb2gray
cv2.GaussianBlur(img,(5,5),0)	Gaussian blur ลด noise ก่อน threshold
cv2.threshold(THRESH_OTSU)	Otsu threshold อัตโนมัติ — เทียบเท่า graythresh+im2bw
cv2.connectedComponentsWithStats()	หา connected components — เทียบเท่า bwlabel
cv2.findContours(...)	ค้นหา contour จาก box_mask เพื่อหากล่องคำตอบ (ใช้ร่วมกับ Morphological Line Detection)
cv2.morphologyEx(MORPH_OPEN)	สกัดเส้นแนวนอน/ตั้ง — ใช้ใน Morphological Line Detection ทั้ง 3 engine
cv2.morphologyEx(MORPH_CLOSE)	เชื่อมมุมกล่องให้สนิท
cv2.resize(img,(W,H))	Resize ภาพ — เทียบเท่า imresize([42 24])
cv2.copyMakeBorder(...)	เพิ่ม padding สีขาวรอบ ROI ก่อนส่ง OCR
cv2.matchTemplate(TM_CCOEFF_NORMED)	Normalized cross-correlation — corr2
np.mean(A)	ค่าเฉลี่ย — ใช้คำนวณ InputAVG, PatternAVG ใน corr2
np.sum((A-Am)*(B-Bm))	Numerator ของสูตร corr2
ImageFont.truetype(path,size)	โหลด TrueType font สร้าง digit template
pytesseract.image_to_string()	อ่านข้อความด้วย Tesseract LSTM Engine
--oem 3 --psm 7	Tesseract: LSTM engine + single line mode
TrOCRProcessor.from_pretrained()	โหลด preprocessor สำหรับ TrOCR
VisionEncoderDecoderModel()	โหลด Vision Transformer model ของ Microsoft

model.generate(pixel_values)	ให้ Transformer อ่านข้อความจากภาพ
processor.batch_decode(ids)	แปลง token ids กลับเป็นข้อความ
render_template(name,**ctx)	แสดง HTML ผลลัพธ์ผ่าน Flask
corr2(A,B)	คำนวณ 2D Correlation — หัวใจของวิธีที่ 1
create_templates()	สร้าง template 0–9 — เทียบเท่า create_templates.m
extract_answers(path)	Public API: อ่านคำตอบทุกกล่อง (ทั้ง 3 วิธี)
grade(image_path, provided_key)	ตรวจการบ้าน: เปรียบเทียบกับเฉลย คืนคะแนน — รองรับ provided_key (กรอกจากหน้าเว็บ) และ answer_key.json